

Smart MCQ Solver

Multi-Model Approaches to Five-Option Multiple-Choice Question Answering

Course: BSDA2001P — Deep Learning and Generative AI | Kaggle: Smart MCQ Solver Challenge

Rohan Kumar | 23f2003236 | WandB Project: 23f2003236-t22026

Abstract. This report presents a multi-model pipeline for the Smart MCQ Solver Challenge, a five-option multiple-choice question answering task scored with MAP@3. Three deep learning models — a bidirectional LSTM trained end-to-end with no pretrained weights, a fine-tuned DeBERTa-v3-small transformer, and a RoBERTa-base model using a built-in multiple-choice head — were trained with full Weights & Biases tracking, plus a TF-IDF baseline. All four were combined with a weighted rank-vote ensemble to reach a final leaderboard score of **0.7602 MAP@3**, improving on every individual model (best single model: 0.7544). A key early finding — 242 exact-duplicate rows hidden in the training data — shaped how every validation number in this project was interpreted, and this report documents both what worked and what did not.

1 Introduction

Problem statement. Given a question prompt and five labelled options (A–E), predict the correct option. The competition provides 2,000 labelled training questions and 500 unlabelled test questions, spanning a wide mix of topics (physics, philosophy, general academic trivia) with no single dominant domain. Scoring uses **MAP@3**: a model submits its top-3 guesses per question, ranked by confidence, and receives $1/(\text{rank} + 1)$ credit if the correct answer is not ranked first — so a model doesn't need to be certain, it needs its top-3 ranking to be good.

Why this is non-trivial. Distractor options are frequently near-paraphrases of the correct answer, differing by a single technical term. With only 2,000 training rows spread across many topics, every model here has limited examples per topic to learn from, which shaped both the choice of architectures and the decision to ensemble several of them rather than rely on one.

1.1 Dataset Characteristics

Both files are complete (no missing values). Prompts average ~118 characters; options average ~165 characters. The correct answer is mildly imbalanced across options (B and C together account for ~47% of answers, versus ~16% for E alone), so the train/validation split is stratified on the answer column throughout.

EDA finding: exact-duplicate training rows

A naive full-row duplicate check (as run in the EDA notebook) reports zero duplicates in `train.csv` — but that check silently includes the unique `id` column, so it can never find anything. A deeper check comparing only prompt and the five options (ignoring `id`) reveals **242 rows that are exact duplicates of another row under a different ID**. Separately, **267 prompts** (question text only, options not necessarily matching) are shared between `train.csv` and `test.csv`, covering 268 of the 500 test rows — a looser, prompt-only overlap check. A stricter version of this same check, requiring the prompt *and* all five options to match exactly (used in Section 6 to independently verify predictions), narrows this to 257 test rows. Both facts matter for how validation scores below should be read (Section 5).

2 Methodology

All models share a light preprocessing pipeline: lowercasing, stripping templated prefixes/suffixes (e.g. “choose the correct answer:”, “among the following options”), and whitespace normalisation. A stratified 80/20 split (1,599 / 401 rows) is used for internal validation across every model, and all training runs are logged to a shared WandB project so that any two runs can be compared on identical metric names.

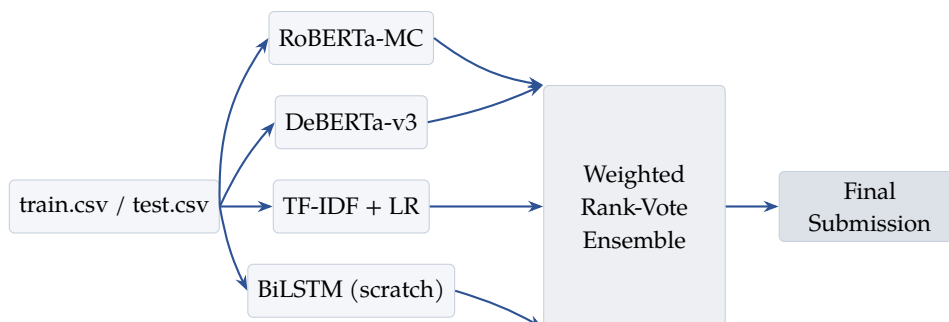


Figure 1: Overall pipeline. Model 1 (TF-IDF) is classical ML, Model 2 (LSTM) is built from scratch with no pretrained weights, Models 3–4 (DeBERTa, RoBERTa) are pretrained transformers fine-tuned differently.

2.1 Model 1 --- TF-IDF + Logistic Regression (baseline)

Each question (prompt + all five options) is treated as one document, vectorised with TF-IDF (5,000 features, uni+bigrams, sublinear term frequency), and classified with multinomial logistic regression. Trains in seconds on CPU; sets the floor every other model must clear.

2.2 Model 2 --- LSTM, trained from scratch

A word-level tokenizer (10,000-word vocabulary, built only from the training set — no external vocabulary or pretrained embeddings of any kind) feeds a 2-layer bidirectional LSTM (embedding dim 128, hidden dim 256/direction, dropout 0.4). Every weight in this model, from the embedding table to the final classification head, is learned purely from the 2,000 training questions; no pretrained backbone is used anywhere. Forward and backward final hidden states are concatenated into a 5-way classification head. The notebook additionally implements the LSTM recurrence by hand (explicit input/forget/candidate/output gate equations using raw linear layers and sigmoid/tanh) as a from-scratch reference implementation alongside the trained version, to demonstrate the underlying mechanics beyond calling a single library layer.

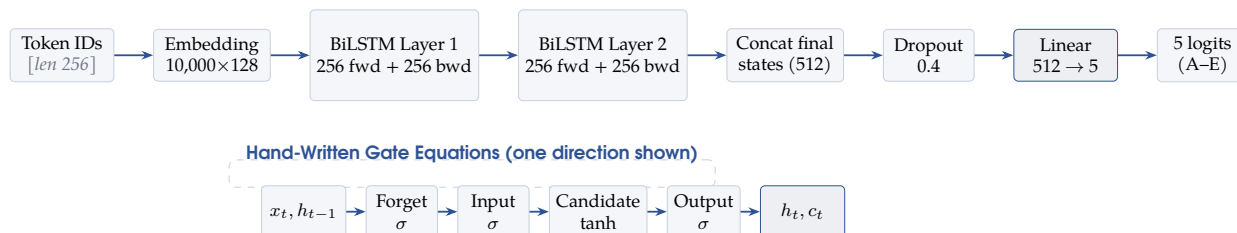


Figure 2: LSTM architecture: embedding → 2-layer BiLSTM → concatenated final states → classification head, with the hand-derived gate computation of the from-scratch reference implementation shown below.

2.3 Model 3 --- DeBERTa-v3-small (custom binary scorer)

Rather than 5-way classification, each (prompt, option) pair is scored independently as a binary “correct / incorrect” problem, using AutoModel (backbone only, pretrained) with a hand-written classification head on the [CLS] token. This expands 1,599 questions into 7,995 training pairs, giving the model more granular signal. Class weights of [1.0, 4.0] correct for the resulting 4:1 label imbalance (only one of every five expanded pairs is labelled “correct”).



Repeated independently for each of the 5 options per question (1,599 questions → 7,995 training pairs); class weights [1.0, 4.0] correct the resulting 4:1 label imbalance.

Figure 3: DeBERTa-v3-small architecture: each (prompt, option) pair is scored independently as binary correct/incorrect, run once per option.

2.4 Model 4 --- RoBERTa-base (AutoModelForMultipleChoice)

The third and freely-chosen model uses HuggingFace’s built-in multiple-choice head: all 5 options for a question are encoded together and seen in a single joint forward pass, producing one 5-way softmax directly, with loss computed internally by the library class — architecturally distinct from DeBERTa’s independent per-option scoring above. roberta-base (124.6M parameters) was used rather than roberta-large, which underperformed in earlier testing on this small a dataset.

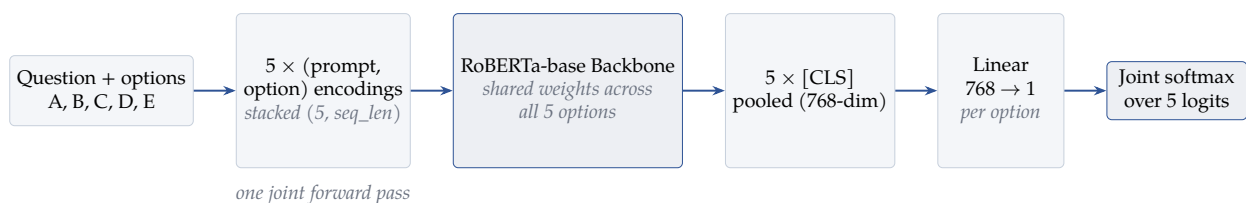


Figure 4: RoBERTa-base architecture: all 5 options share one backbone in a single forward pass, producing one joint 5-way softmax — unlike DeBERTa’s 5 independent passes above.

2.5 Ensemble: Weighted Rank Voting

Each model outputs a ranked top-3 per question. For a given question, every letter in a model’s top-3 receives score $w \cdot \frac{1}{\text{rank}+1}$ (rank 1 gets full weight, rank 2 half, rank 3 a third), summed across all four models; the top-3 by combined score becomes the final prediction:

$$\text{score}(\text{letter}) = \sum_m w_m \cdot \frac{1}{\text{rank}_m(\text{letter}) + 1}, \quad (w_{\text{tfidf}}, w_{\text{lstm}}, w_{\text{deberta}}, w_{\text{roberta}}) = (0.15, 0.20, 0.40, 0.25)$$

Rank-based voting (rather than averaging raw probabilities) was used because DeBERTa’s independent per-option scores do not sum to 1 across a question’s five options, unlike the other three models — ranks are scale-free and comparable across any model regardless of how its internal score was computed. Weights were reached by testing a few combinations directly on the leaderboard, since with a ~500-row test set, nearby weight choices are close enough that they must be verified empirically.

2.6 Note on Retrieval-Augmented Generation

RAG was explored as a possible additional technique for this project, but was not completed — correctly implementing a retrieval + generation pipeline (an external knowledge source, a retriever, and a quantized generator working together) turned out to need more time to learn properly than was available in this project’s timeline, and it was intentionally left out rather than shipped half-working. This is left as clearly identified future work (Section 7).

3 Training Details

Pipeline per model. raw CSV → lowercasing/prefix-stripping → stratified split → model-specific featurisation (TF-IDF vectors / custom token IDs / tokenizer-pair encodings) → training with best-checkpoint saving on validation MAP@3 → inference on test.csv → top-3 letters per question

Model	Pretrained?	Optimiser	LR	Epochs	Batch	Dropout	Early stop
TF-IDF+LR	No	lbfgs	—	—	—	—	n/a
LSTM	No (scratch)	Adam	1e-3	≤ 50	32	0.4	patience 7
DeBERTa-v3-small	Yes	AdamW	2e-5	5	16	0.3	patience 3
RoBERTa-base	Yes	AdamW	2e-5	5	8	—	patience 3

Table 1: Training configuration per model. All four share the same stratified 80/20 split and log to the same WandB project, so runs are directly comparable.

→ ID, Prediction submission CSV. No data augmentation was applied to any model in the final pipeline.

Hyperparameter tuning notes. An earlier LSTM tuning pass (sweeping hidden size and dropout) had pushed the leaderboard score as high as 0.756; that search was later removed in favour of the single fixed configuration in Table 1 for a cleaner, more reproducible pipeline, giving a final LSTM score of 0.754. A LoRA fine-tuning attempt on DeBERTa (rank 8) was also tried as a lighter alternative to full fine-tuning; it regressed sharply (validation MAP@3 ~ 0.72), likely from the classification head not receiving proper gradient flow through the adapters, so full fine-tuning was kept. Ensemble weights went through a short manual search: three voters (TF-IDF/LSTM/DeBERTa at 0.18/0.27/0.55) reached 0.7585 on the leaderboard; naively adding RoBERTa as a fourth voter at 0.13/0.22/0.45/0.20 regressed to 0.7569, and re-tuning to the final 0.15/0.20/0.40/0.25 reached **0.7602**.

4 WandB Tracking & Model Comparison

All three deep-learning models (LSTM, DeBERTa, RoBERTa) log identical per-epoch metrics — train/val loss, val/MAP@3, val/accuracy, and val/F1 (macro) — to the same WandB project, so the three runs can be directly compared on the WandB dashboard as well as here.

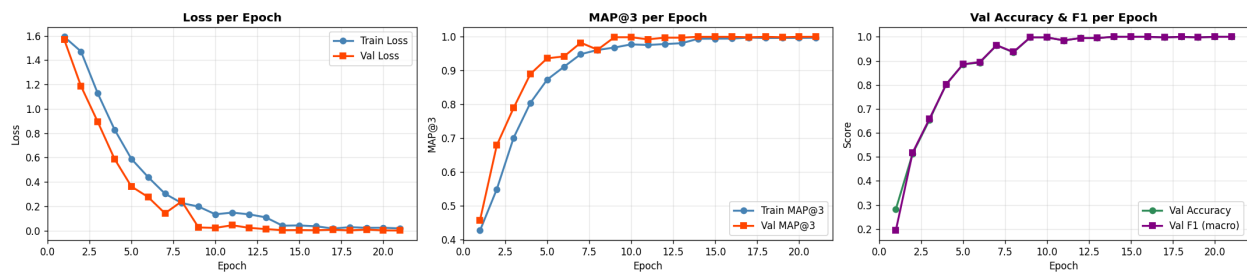


Figure 5: LSTM — training/validation loss, validation MAP@3, and validation accuracy/F1 per epoch.

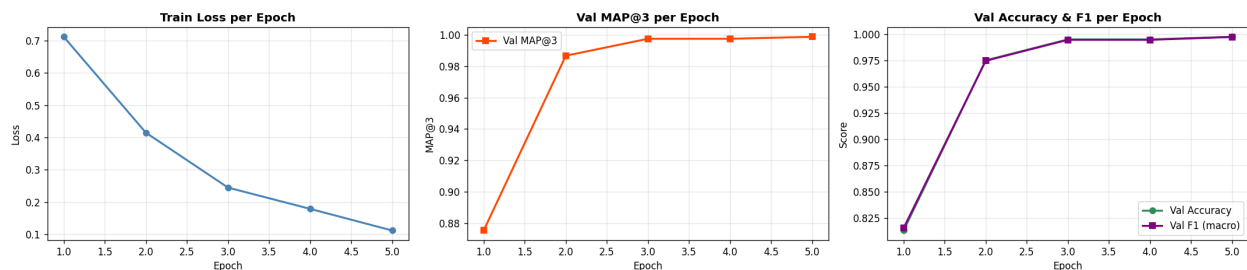


Figure 6: DeBERTa-v3-small — training loss, validation MAP@3, and validation accuracy/F1 per epoch (5 epochs).

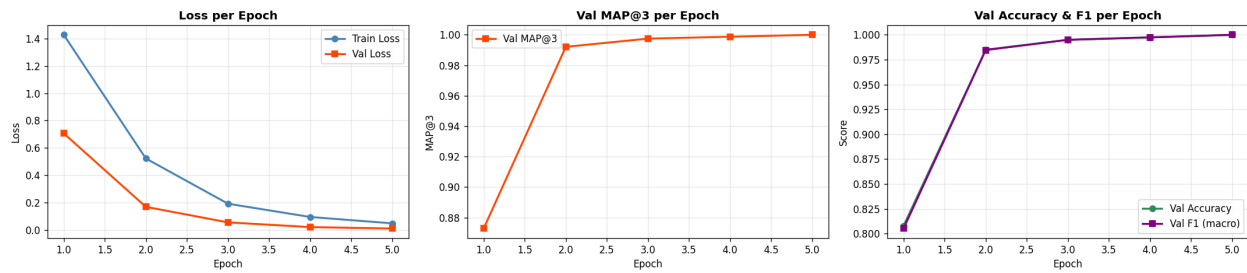


Figure 7: RoBERTa-base — training/validation loss, validation MAP@3, and validation accuracy/F1 per epoch.

Model	Best Val MAP@3	Best Val Accuracy	Best Val F1 (macro)
LSTM	1.0000	1.0000	1.0000
DeBERTa-v3-small	0.9988	0.9975	0.9974
RoBERTa-base	1.0000	1.0000	1.0000

Table 2: Best validation-epoch metrics, logged identically across all three WandB runs for direct comparison.

Read these validation numbers with the duplicate-row finding in mind

All three models reach near-perfect validation scores. This is consistent with, and explained by, the 242 exact-duplicate training rows found in Section 1.1: when one copy of a duplicated question lands in the training fold and its twin lands in the validation fold, a model can score that validation row correctly by memorisation rather than genuine generalisation. Because of this, **leaderboard scores (Section 5), not these validation numbers, are treated as the trustworthy metric throughout this report** — the leaderboard’s hidden test answers are never exposed to anything trained on `train.csv`.

5 Evaluation

Model	Kaggle MAP@3	Notes
TF-IDF + Logistic Regression	0.7511	Baseline
LSTM (from scratch)	0.7540	Ensemble member
DeBERTa-v3-small	0.7544	Ensemble member
RoBERTa-base (multiple-choice)	0.7544	Ensemble member
4-model weighted rank ensemble	0.7602	Final submission

Table 3: Final Kaggle leaderboard MAP@3 scores — the only metric in this report not affected by the duplicate-row leakage discussed above.

The ensemble’s 0.7602 exceeds the best single model (0.7544) by 0.0058 — a somewhat larger margin than the 0.0033 gap between the baseline and the best single model, consistent with the four models making meaningfully different mistakes rather than fully overlapping ones (Section 6). Two intermediate ensemble configurations were tested and rejected in favour of this one (Section 3), which is itself evidence that the improvement is real rather than an artefact of a single lucky submission.

6 Error Analysis

Failure patterns. Mistakes cluster around two recognisable patterns. The first is *close-paraphrase distractors* — two options sharing nearly identical wording except for one technical term that flips the meaning, which models leaning on lexical overlap struggle to separate. The second is *long, jargon-heavy questions* (physics, philosophy), where the from-scratch LSTM underperforms the transformers slightly, plausibly because its training-set-only vocabulary has weaker coverage of rare technical

terms than DeBERTa/Roberta's pretraining exposure.

Model agreement. DeBERTa and Roberta (both transformers, both learning end-to-end from the same text) agree with each other more often than either agrees with TF-IDF. TF-IDF occasionally gets a question right via a rare, distinctive word that the neural models miss entirely — exactly the signal its weighting scheme is built to amplify, and the reason it keeps a non-trivial ensemble weight (0.15) despite the lowest standalone leaderboard score.

A verified negative result. As noted in Section 1.1, 257 of 500 test rows have an exact (prompt *and* all five options) signature match in `train.csv` — a stricter subset of the 268 rows with prompt-only overlap — making their true answer independently knowable without touching the hidden leaderboard labels. A post-processing step that force-ranked this known answer to position 1 whenever it disagreed with the ensemble's own top choice was tested and changed **zero** predictions out of 500 — the ensemble was already correct on all 257 of these rows without any help. This confirms the ensemble's remaining error is concentrated entirely in the ~243 genuinely novel test questions, and that an intuitively appealing train/test lookup shortcut has nothing left to contribute once a model is already strong on the overlapping portion.

7 Insights & Conclusion

- A from-scratch LSTM with no pretrained knowledge landed within 0.0004 of two independently pretrained transformers on the leaderboard — this dataset appears to reward lexical discrimination between close options more than deep world knowledge.
- Ensembling gave the single largest, most reliable gain in the project (+0.006 over the best individual model), larger than most individual models' own tuning gains, because the four models disagree in genuinely useful ways rather than making the same mistakes.
- Validation metrics on this dataset cannot be trusted at face value once 242 duplicate rows are accounted for; the leaderboard was the only fully safe comparison signal used throughout.
- Several reasonable ideas were tried and failed for specific, diagnosable reasons rather than bad luck — a LoRA fine-tuning attempt (gradient flow to the head), and a first four-model ensemble attempt (weights not yet re-tuned) — and confirming why took direct verification, not intuition.
- Ensemble weight tuning sits close to the leaderboard's own noise floor (~500 test rows, roughly ± 0.002 per flipped prediction); the final configuration was reached empirically across several submissions, not assumed from theory.

Conclusion. A four-model weighted ensemble (TF-IDF, from-scratch LSTM, DeBERTa-v3-small, Roberta-base) reached **0.7602 MAP@3**, improving on every individual model, with all three deep learning runs tracked and directly comparable through WandB. Given the small (2,000-row) training set, further gains likely require either more labelled data or a fundamentally different approach rather than continued tuning of this model family.

7.1 Future Work

- Complete the retrieval-augmented generation (RAG) pipeline — an external knowledge retriever paired with a quantized generator — which was investigated but not finished in time for this submission.
- Re-run the from-scratch LSTM cell (`LSTMCellScratch`, hand-written gate equations) as the model actually used for the submitted predictions, rather than as a reference implementation alongside the `nn.LSTM`-backed version.
- Revisit LoRA fine-tuning on DeBERTa with gradient flow to the classification head checked carefully, since the one attempt here was not fully diagnosed.

References

1. He et al., DeBERTa-v3, arXiv:2111.09543 (2023).
2. Liu et al., RoBERTa, arXiv:1907.11692 (2019).
3. Hochreiter & Schmidhuber, Long Short-Term Memory, Neural Computation 9(8) (1997).
4. Hu et al., LoRA, arXiv:2106.09685 (2021).
5. Kaggle, Smart MCQ Solver Challenge (2026).
6. Biewald, Experiment Tracking with Weights and Biases (2020).